

PostgreSQL Streaming Replication

Relocating the Standby to the STL Zone — New VM, New IP

INFO — Scope and starting point

Re-home the two-node streaming-replication standby onto a fresh VM in the **STL** zone (new hostname, new IP), retiring the current **YKT** standby. This runbook **assumes db03 has already been built by your node-build automation** with a PostgreSQL version matching db01 — so OS, packages, directory layout and SELinux are done. It picks up at replication readiness and carries through cutover, cross-checks, and decommissioning. The new standby is built **alongside** the running YKT node using a temporary slot, so client downtime is limited to a short slot swap plus validation.

Topology — before and after

Before: db01 (primary, STL) → db02 (standby, YKT) via slot `ssncpri`.

After: db01 (primary, STL) → db03 (standby, **STL**) via slot `ssncpri`. db02/YKT decommissioned.

db01	Primary — STL zone (unchanged throughout)
db02	Current standby — YKT zone (to be retired)
db03	New standby — STL zone; automation-built, version-matched to db01
Replication user	replicator (REPLICATION, LOGIN)
Canonical slot	ssncpri (final slot the new standby will consume)
Temporary slot	ssncpri_stl (used only during the parallel build)
application_name	db03 (set via the base-backup conninfo — see §5)

1. Set your working variables

Define these once per shell session. Run the primary block on db01 and the standby block on db03.

db01 — primary-side variables

```
# ---- On db01 (PRIMARY, STL) ----
export PGVER=16 # match your actual major version
export PGBIN=/usr/pgsql-{PGVER}/bin
export PGDATA=/apps/pgsql_data/{PGVER}
export PGPORT=5432
export SLOT=ssncpri # canonical slot
export TMP SLOT=ssncpri_stl # temp slot for the parallel build
export NEW_STANDBY_IP=10.x.x.x # db03 IP (STL)
export OLD_STANDBY_IP=10.y.y.y # db02 IP (YKT), for hba cleanup
```

db03 — standby-side variables

```
# ---- On db03 (NEW STANDBY, STL) ----
export PGVER=16
export PGBIN=/usr/pgsql-{PGVER}/bin
export PGDATA=/apps/pgsql_data/{PGVER}
export PGPORT=5432
export PRIMARY_HOST=db01 # db01 IP/FQDN reachable from STL
export REPL_USER=replicator
export APPNAME=db03 # shows in pg_stat_replication / sync names
export SLOT=ssncpri
export TMP SLOT=ssncpri_stl
```

2. Pre-flight checks on db01 (primary)

2.1 Version, current replication, slot health

db01 — inspect replication and slot state

```
{PGBIN}/postgres --version # db03 must match this major.minor exactly

# Current replication — db02/YKT should be streaming
sudo -u postgres psql -x -c "SELECT application_name, client_addr, state,
sync_state, sent_lsn, replay_lsn, replay_lag FROM pg_stat_replication;"

# Slot health — ssncpri active, wal_status must NOT be 'lost'
sudo -u postgres psql -x -c "SELECT slot_name, active, active_pid,
wal_status, restart_lsn FROM pg_replication_slots;"
```

2.2 WAL retention and archiving

db01 — WAL retention / archive settings

```
sudo -u postgres psql -c "SHOW max_slot_wal_keep_size;" # -1 = unlimited (safest)
sudo -u postgres psql -c "SHOW wal_keep_size;"
sudo -u postgres psql -c "SHOW archive_mode;"
sudo -u postgres psql -c "SHOW archive_command;"
```

2.3 Replication topology settings

db01 — level, sender headroom, sync config

```
sudo -u postgres psql -c "SHOW wal_level;" # replica or logical
sudo -u postgres psql -c "SHOW max_wal_senders;" # headroom for a 2nd standby
sudo -u postgres psql -c "SHOW synchronous_standby_names;" # BLANK = async (see note)
```

CAUTION — If synchronous replication is in use

A non-blank `synchronous_standby_names` means commits wait on a sync standby. Do **not** simply stop db02 at cutover — that can stall commits on db01. The correct ordering is handled in §7.3: add db03 to the sync set and confirm it before removing db02. If the value is blank, replication is async and you can ignore this.

3. Ready the automation-built db03 for replication

db03 already exists as a standalone, version-matched node. Before it can become a standby, cross-check that it truly matches db01, then quiesce it and empty \$PGDATA so the base backup can populate it.

3.1 Cross-check db03 against db01

Run each side and confirm the pair matches. Encoding and collation matter even though the base backup later overwrites the cluster — the underlying OS collation libraries must agree or indexes can drift.

Check	How to verify	Expected
Major.minor version	\$PGBIN/postgres --version on both	Identical, incl. minor
Server encoding	psql -c "SHOW server_encoding"	Identical
Collation / ctype	psql -c "SHOW lc_collate" and lc_ctype	Identical
glibc / ICU version	ldd --version; psql -c "SHOW icu_version" if ICU	Same as db01
Data path / port	echo \$PGDATA; psql -c "SHOW port"	Match convention
Clock / NTP	timedatectl status; chrony synced	Synced, same TZ
Filesystem headroom	df -h /apps/pgsql_data	≥ primary data size + margin

3.2 Stop the standalone instance and empty \$PGDATA

pg_basebackup requires an empty target directory. Stop the instance the automation started and clear the cluster.

db03 — quiesce and clear the target cluster

```
# On db03
hostname # CONFIRM this is db03, not db01/db02
sudo systemctl stop postgresql-${PGVER}
ps -u postgres -o pid,cmd | grep -i postgres # verify nothing is running

# Move the auto-built cluster aside (reversible) rather than rm -rf
sudo -u postgres bash -c 'mv "$PGDATA" "${PGDATA}.autobuilt_$(date +%Y%m%d_%H%M) "'
sudo -u postgres install -d -m 0700 "$PGDATA" # empty dir, correct perms
```

CRITICAL — Confirm the host before clearing

The hostname check is deliberate — clearing \$PGDATA on the wrong node is unrecoverable. Because db03 is a fresh automation build with no real data, moving the cluster aside (not deleting) keeps a clean fallback until db03 is verified healthy.

3.3 Replication credential for the replicator user

So the secret never lands in `postgresql.auto.conf`, give the postgres OS user a `.pgpass` entry (skip if your automation already provisions this).

db03 — .pgpass for replicator

```
# On db03, as postgres
umask 077
echo 'db01:5432:replication:replicator:<REPL_PASSWORD>' >> ~/.pgpass
chmod 0600 ~/.pgpass
```

3.4 Connectivity pre-flight to db01

Prove db03 can open a replication connection **before** the base backup, so a firewall or pg_hba gap surfaces now, not mid-copy. (Section 4 adds the hba line first; run this immediately after.)

db03 — verify replication connectivity

```
# On db03 — TCP reachability
timeout 5 bash -c "</dev/tcp/${PRIMARY_HOST}/${PGPORT}" && echo 'port open'

# Replication-mode login test (should connect, then quit)
$PGBIN/psql "host=${PRIMARY_HOST} port=${PGPORT} user=${REPL_USER} \
dbname=replication replication=database" -c "IDENTIFY_SYSTEM;"
```

4. Authorise db03 on db01 (primary)

Add a replication rule for the new STL IP. Leave the existing YKT line in place — you remove it only at cutover. Match the auth method your existing YKT line uses (scram-sha-256 or md5).

db01 — authorise db03 and verify the role

```
# On db01, as postgres
grep -n replication $PGDATA/pg_hba.conf # note the method on the YKT line

cat >> $PGDATA/pg_hba.conf <<HBA
# db03 (STL) replication — standby relocation
host replication replicator ${NEW_STANDBY_IP}/32 scram-sha-256
HBA

sudo -u postgres psql -c "SELECT pg_reload_conf();" # reload; no restart needed

# Confirm the replicator role can log in and replicate
sudo -u postgres psql -c "SELECT rolname, rolreplication, rolcanlogin
FROM pg_roles WHERE rolname = 'replicator';" # both must be t
```

5. Build db03 with a temporary slot

Run the base backup on db03 against a **new temporary slot** so it coexists with db02 (still owner of ssncpri). Passing `application_name=db03` in the conninfo means `-R` writes it into `primary_conninfo`, giving a clean identity in `pg_stat_replication` and sync sets.

db03 — base backup against the temporary slot

```
# On db03, as postgres. $PGDATA must be EMPTY (done in 3.2).
$PGBIN/pg_basebackup \
-d "host=${PRIMARY_HOST} port=${PGPORT} user=${REPL_USER} \
application_name=${APPNAME}" \
--pgdata=$PGDATA \
--wal-method=stream \ # -Xs: self-consistent copy
--create-slot --slot=$TMP_SLOT \ # create + consume ssncpri_stl
--write-recovery-conf \ # -R: standby.signal + conninfo + slot name
--checkpoint=fast \ # don't wait on a spread checkpoint
--progress --verbose
```

5.1 Verify the recovery wiring before starting

db03 — confirm standby wiring

```
ls -l $PGDATA/standby.signal # must exist
grep -E 'primary_conninfo|primary_slot_name' \
$PGDATA/postgresql.auto.conf
# expect: primary_slot_name = 'ssncpri_stl'
# expect: application_name=db03 inside primary_conninfo
```

NOTE — Config comes along for free

On the PGDG RPM layout, `postgresql.conf` lives inside `$PGDATA`, so the base backup copies db01's settings. The replication-sensitive parameters (`max_connections`, `max_worker_processes`, `max_locks_per_transaction`, `max_prepared_transactions`) already match — no standby-first tuning is needed. Re-check only host-specific values (e.g. `shared_buffers`) if db03 is sized differently from db01.

6. Start db03 and verify it streams

db03 / db01 — start and confirm streaming

```
# On db03
sudo systemctl enable --now postgresql-${PGVER}
sudo -u postgres psql -c "SELECT pg_is_in_recovery();" # t
sudo -u postgres psql -x -c "SELECT status, sender_host, slot_name,
latest_end_lsn FROM pg_stat_wal_receiver;" # status = streaming

# On db01 — BOTH standbys should now be streaming
sudo -u postgres psql -x -c "SELECT application_name, client_addr, state,
replay_lag FROM pg_stat_replication ORDER BY application_name;"
```

6.1 Gate check — do not proceed until all pass

Check	How to verify	Expected
db03 in recovery	pg_is_in_recovery() on db03	t
WAL receiver up	pg_stat_wal_receiver.status	streaming
Slot on temp	slot_name on db03	ssncpri_stl
Two standbys	rows in pg_stat_replication on db01	db02 and db03
db03 caught up	replay_lag for db03	~0

SUCCESS — Checkpoint — no downtime yet

db03 is a fully caught-up standby running in parallel with db02, which stays as your live fallback. Nothing client-facing has changed; the maintenance window has not started.

7. Cutover during the downtime window

7.1 Confirm db03 is at zero lag

db01 — verify db03 fully caught up

```
sudo -u postgres psql -x -c "SELECT application_name, client_addr, replay_lag
FROM pg_stat_replication;" # db03 replay_lag ~0
```

7.2 (Async setups) stop the YKT standby

If synchronous_standby_names was blank in §2.3, stop db02 now. If sync was in use, do §7.3 first.

db02 — stop the old standby (async path)

```
# On db02 (YKT)
sudo systemctl stop postgresql-${PGVER}
# ssncpri goes inactive on db01 but is RETAINED — do not drop it yet
```


7.3 (Sync setups) add db03 to the sync set before removing db02

For synchronous replication, make db03 a valid sync target and confirm it **before** stopping db02, so commits never stall on a missing sync standby.

db01 — sync-safe cutover ordering

```
# On db01 — include db03 (adjust to your quorum expression)
sudo -u postgres psql -c "ALTER SYSTEM SET synchronous_standby_names
= 'ANY 1 (db02, db03)';"
sudo -u postgres psql -c "SELECT pg_reload_conf();"

# Confirm db03 is a live sync candidate, THEN stop db02
sudo -u postgres psql -x -c "SELECT application_name, sync_state
FROM pg_stat_replication;" # db03 sync or potential
sudo systemctl -H db02 stop postgresql-${PGVER} # or run locally on db02

# Narrow the sync set to db03 only
sudo -u postgres psql -c "ALTER SYSTEM SET synchronous_standby_names = 'db03';"
sudo -u postgres psql -c "SELECT pg_reload_conf();"
```

7.4 Move db03 onto the canonical slot

Since PostgreSQL 13, `primary_slot_name` applies on reload — the WAL receiver restarts on the new slot, no full bounce. Use a targeted `ALTER SYSTEM SET` (never `RESET ALL` on a standby).

db03 — switch `primary_slot_name` to `ssncpri`

```
# On db03
sudo -u postgres psql -c "ALTER SYSTEM SET primary_slot_name = 'ssncpri';"
sudo -u postgres psql -c "SELECT pg_reload_conf();"
sudo -u postgres psql -x -c "SELECT status, slot_name
FROM pg_stat_wal_receiver;" # slot_name = ssncpri
```

7.5 Verify on db01, then clean up slots and hba

db01 — confirm, drop temp slot, prune hba

```
# On db01 — ssncpri active again, now consumed by db03
sudo -u postgres psql -x -c "SELECT slot_name, active, active_pid, wal_status
FROM pg_replication_slots ORDER BY slot_name;"

# ONLY after ssncpri shows active=t via db03: drop the temp slot
sudo -u postgres psql -c "SELECT pg_drop_replication_slot('ssncpri_stl');"

# Remove the retired YKT replication line, then reload
sed -i.bak "${OLD_STANDBY_IP//./\\.}/32/d" $PGDATA/pg_hba.conf
sudo -u postgres psql -c "SELECT pg_reload_conf();"
```

CRITICAL — Ordering is the safety net

Never drop `ssncpri_stl` or the db02 hba line until db03 is confirmed streaming on `ssncpri`. Until then, restarting db02 (it reclaims `ssncpri`) or reverting db03 to the temp slot are both valid rollbacks (\$9).

8. Post-cutover validation

db01 / db03 — end-to-end replication proof

```
# On db01 — write a sentinel
sudo -u postgres psql -c "CREATE TABLE IF NOT EXISTS repl_check(id int, ts
timestamp);"
sudo -u postgres psql -c "INSERT INTO repl_check VALUES (1, now());"

# On db03 — the row must appear (read-only standby)
sudo -u postgres psql -c "SELECT * FROM repl_check;"
sudo -u postgres psql -c "SELECT now() - pg_last_xact_replay_timestamp() AS
replay_lag;"

# On db01 — final state: one standby (db03) on ssncpri
sudo -u postgres psql -x -c "SELECT application_name, client_addr, state,
sync_state, replay_lag FROM pg_stat_replication;"
```

8.1 Done criteria

Check	How to verify	Expected
Round trip	sentinel row visible on db03	1 row returned
Lag	replay_lag on db03	~0
Single standby	pg_stat_replication on db01	db03 only
Slot	ssncpri active; ssncpri_stl gone	active=t; absent
Timeline	pg_control_checkpoint() TLI on db01 vs db03	Same TLI
Sync (if used)	sync_state for db03	sync

SUCCESS — Window can close

When every row above passes, the relocation is complete and the maintenance window can be closed. Keep db02 powered (PostgreSQL stopped) as a fallback until you are fully confident.

9. Rollback plan

- **Before cutover (build/QA fails):** nothing client-facing changed; db02/YKT is still live. Drop ssncpri_stl on db01 and retry.
- **During cutover (db03 won't take ssncpri):** revert db03 with ALTER SYSTEM SET primary_slot_name='ssncpri_stl' + reload, or restart db02 — it reclaims ssncpri since the slot was never dropped. Restore synchronous_standby_names to its original value if you changed it.
- **After validation:** the moved-aside \${PGDATA}.autobuilt_* on db03 can be removed once db03 is proven healthy.

10. Decommission YKT and housekeeping

- On db02: `systemctl disable --now postgresql- $\{PGVER\}$` , then release the VM per your infra process.
- Update inventory, DNS, and Ansible/AWX host groups so db03 replaces db02.
- Point monitoring (fleet pack) at db03 and drop db02 alerts; add a `replay_lag` and `slot wal_status` alert.
- Confirm backups (`pg_basebackup` / `pgBackRest`) reference db03 where relevant and that `/apps/pgsql_archives` retention is unchanged.
- Record the change and archive this runbook.

Appendix A — Verification query pack

Verification query pack

```
-- PRIMARY (db01): replication + slots
SELECT application_name, client_addr, state, sync_state,
sent_lsn, replay_lsn, replay_lag FROM pg_stat_replication;
SELECT slot_name, active, active_pid, wal_status, restart_lsn
FROM pg_replication_slots;
SELECT timeline_id FROM pg_control_checkpoint();

-- STANDBY (db03)
SELECT pg_is_in_recovery();
SELECT status, sender_host, slot_name, latest_end_lsn FROM pg_stat_wal_receiver;
SELECT now() - pg_last_xact_replay_timestamp() AS replay_lag;
SELECT timeline_id FROM pg_control_checkpoint();
```

Appendix B — Important things to keep in mind

- **Empty \$PGDATA is mandatory** for pg_basebackup; the automation-built cluster must be stopped and cleared first (§3.2).
- **Synchronous replication ordering** — if synchronous_standby_names is set, add db03 to the sync set and confirm it *before* removing db02, or commits on db01 can stall (§7.3).
- **Collation drift** — even though the base backup overwrites the cluster, db03's OS glibc/ICU must match db01's, or collation-dependent indexes can misbehave. Your version-matched automation should guarantee this; verify in §3.1.
- **application_name** — set it in the base-backup conninfo so db03 is cleanly identified in pg_stat_replication and any sync quorum expression.
- **One consumer per slot** — a physical slot serves a single active receiver, which is exactly why the parallel build uses ssncpri_stl.
- **Intra-zone latency** — db03 in STL now shares the zone with db01; expect lower network round-trip and steadier replay lag than the old cross-zone YKT link.
- **WAL retention headroom** — the temp slot pins WAL on db01 for the whole build; watch /apps/pgsql_data free space if max_slot_wal_keep_size is finite.
- **Optional safety net** — a restore_command on db03 pointing at a reachable archive lets it recover if it ever falls beyond slot retention; only useful if the archive is reachable from db03.
- **Minor-version match** — db01 and db03 must share major *and* minor or the standby may refuse to start.